

AvaliaRU

Documento de Arquitetura

Versão 1.0

Tabela - Integrantes do Grupo

Matrícula	Nome	Função (responsabilidade)	Pontos de participação na elaboração
251013624	Mateus Alves Araujo	Gestão de projetos e Documentação	
251013787	Murilo Paulino	Back-end	
251040087	Gabriel José Neves de Resende	Analista de Qualidade	
211043772	Vinícius dos Santos Evangelista Dias	Back-end	
251035659	Rafael Lima Sant'Ana	Gestão de projetos e Documentação	
251012313	Luís Gustavo Ferreira Nunes	Back-end	
251037831	Osmar Júnio Mendes Carmo dos Santos	Front-end	
251027835	Rafael Nicolas Freitas Araújo	Análise de Qualidade	
232028428	Luis Henrique Pessato da Mota	Front-end	
232014306	José Gabriel Marques de Araujo	Front-end	
251013651	Matheus Estevam Ferreira	Back-end	
211062428	Rafael Souto Lopes Laube	Back-end	

Histórico de Revisões

Data	Versão	Descrição	Autor(es)
09/05/2026	1.0	Finalização do Documento de Arquitetura	Todos

Sumário

1	<i>Introdução</i>	4
1.1	Propósito	4
1.2	Escopo	4
2	<i>Representação Arquitetural</i>	4
2.1	Definições	4
2.2	Justifique sua escolha.	4
2.3	Detalhamento	5
2.4	Metas e restrições arquiteturais	7
2.5	Visões	8
2.5.1	Visão uso	8
2.5.2	Visão de organização lógica	12
2.5.3	Visão estrutural	12
2.6	Visão de Implantação <relembrem o material da Profa. Milene Serrano>	13
2.7	Restrições adicionais	14
3	<i>Bibliografia</i>	16

1 Introdução

1.1 Propósito

Este documento descreve a arquitetura do sistema sendo desenvolvido pelo grupo, na disciplina de MDS – Métodos de Desenvolvimento de Software – edição do primeiro semestre de 2026, para o sistema AvaliaRU, a fim de fornecer uma visão abrangente do sistema para desenvolvedores, testadores e demais interessados em aspectos relacionados às tecnologias a serem usadas no desenvolvimento.

1.2 Escopo

O detalhamento do escopo se encontra no documento Visão do produto AvaliaRU juntamente com o documento de Visão do produto e do projeto. Porém, em linhas gerais o escopo do produto compreende uma aplicação web que recebe avaliação de pratos e notifica alunos acerca de pratos selecionados e com essas informações faz uma estimativa da lotação do RU, a fim de evitar desperdício de alimentos.

2 Representação Arquitetural

2.1 Definições

A arquitetura do projeto seguirá o modelo Client-Server, no qual existe um servidor central (Server) encarregado de processar as solicitações de acesso e uso do sistema. Nesse servidor, estará instalado, juntamente com o serviço web, o Sistema Gerenciador de Banco de Dados (SGBD) SQLite. O usuário utilizará seu celular ou computador (Cliente) para acessar o sistema e estes dispositivos realizarão as solicitações do usuário diante do servidor.

As trocas de operações entre o cliente e o servidor se darão através do protocolo HTTP e serão encaminhadas através da rede com o protocolo TCP/IP.

A implementação dessa comunicação será realizada utilizando as ferramentas disponíveis no framework Next.js.

2.2 Justificativa

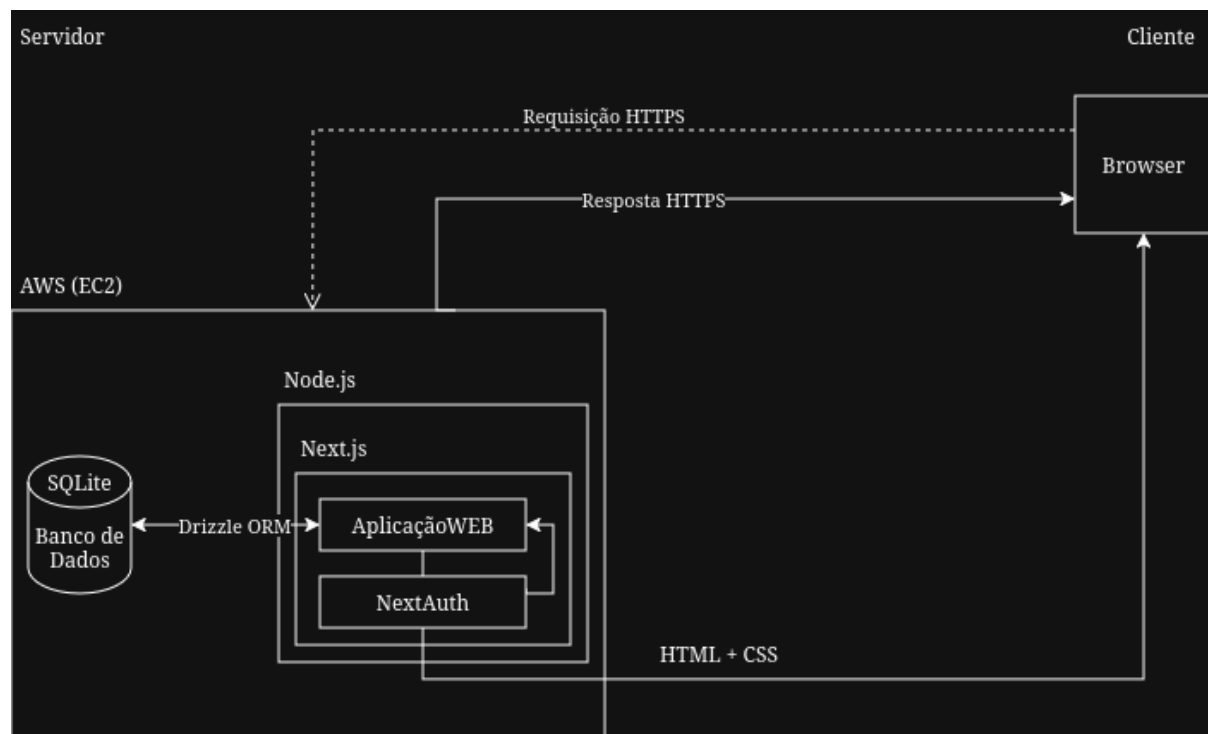
Primeiramente, a arquitetura Client-Server permite separar as responsabilidades diante do funcionamento do sistema. A camada de cliente representa o usuário e é responsável por encaminhar as operações ao servidor central, bem como exibir o resultado do processamento

do servidor. Nesse sentido, o servidor estará encarregado de receber as solicitações dos usuários, processar e encaminhar os resultados para o cliente solicitante.

Em segundo lugar, a arquitetura é amplamente utilizada na web moderna, de modo que a documentação dos softwares utilizados estará disponível com suporte a essa arquitetura.

Em terceiro lugar, nivelamento de conhecimentos da equipe desenvolvedora. Tendo em vista que a arquitetura Client-Server é simples de se compreender, fica evidente que sua implementação também será mais simples, observando fatores como o prazo para entrega, ótima curva de aprendizado do framework Next.js e ampla documentação para ferramentas que deem suporte a essa arquitetura e framework.

2.3 Detalhamento



Cliente (Browser)

O Browser é a interface principal de consumo de dados e interação.

- **Papel:** Renderizar a interface de usuário (UI) e permitir que o Estudante favorite pratos, envie avaliações e consulte métricas públicas.
- **Visualização de Dados:** O estudante tem acesso à média das avaliações de cada prato, permitindo uma escolha informada baseada na opinião da comunidade.
- **Alertas:** Exibe notificações push-up sobre alérgenos críticos e avisa quando um prato favorito será servido no dia.

Gestão de Notificações (Novu)

O serviço Novu (plano gratuito) atua como a infraestrutura de mensagens.

- Responsabilidade: Encaminhar alertas personalizados para o Browser. Os gatilhos incluem avisos de segurança (alergias) e lembretes de conveniência(disponibilidade de pratos favoritos).

Servidor (AWS EC2 / Node.js)

Ambiente de execução centralizado para o processamento de regras complexas.

- Lógica de Match: O Node.js processa o cruzamento diário entre o cardápio e as listas de favoritos dos usuários para agendar os disparos de notificação via Novu.

Framework e Lógica (Next.js / AplicaçãoWEB)

O componente que orquestra a lógica de negócio e a exposição de dados analíticos.

- Métricas de Gestão: Providencia aos funcionários o acesso aos pratos mais favoritos e à média de avaliações, auxiliando no planeamento do cardápio.
- Métricas Públicas: Disponibiliza as médias de satisfação para consulta direta por parte dos estudantes na interface principal.

Segurança (NextAuth)

- Papel: O estudante pode ver a sua própria lista de pratos favoritos, editá-la e receber notificações sobre eles.
- Privacidade: Um estudante A não consegue ver quais são os pratos favoritos do estudante B.
- Segurança (NextAuth): O componente de segurança garante que, quando o servidor busca os favoritos no SQLite, ele filtre apenas aqueles que pertencem ao ID do utilizador que está logado no momento.

Persistência (Drizzle ORM / SQLite)

Camada de dados estruturada para análise em tempo real.

- **Armazenamento:** Guarda as tabelas de Favoritos, Avaliações e perfis de Alergia.
- **Processamento Analítico:** Utiliza o Drizzle ORM para realizar cálculos agregados (médias e contagens) de forma eficiente, servindo tanto o dashboard administrativo quanto a visualização do estudante.

2.4 Metas e restrições arquiteturais

2.4.1 Metas

- **Manutenibilidade e Evolução Independente:** Através da arquitetura Client-Server, busca-se garantir uma forte separação de responsabilidades (Separation of Concerns). A interface de usuário (Client) e a lógica de negócios/dados (Server) devem poder receber manutenção e atualizações de forma independente, sem que uma afete diretamente o funcionamento da outra.
- **Desempenho Otimizado:** O sistema tem como meta fornecer tempos de resposta rápidos e uma navegação fluida. Para isso, o processamento de páginas e requisições pesadas deve ser otimizado através de recursos do Next.js, como a renderização no lado do servidor (SSR), diminuindo a carga de processamento nos dispositivos dos usuários.
- **Produtividade e Nivelamento Técnico:** A arquitetura deve possuir uma curva de aprendizado acessível para garantir o rápido nivelamento técnico da equipe de desenvolvimento. A simplicidade do modelo Client-Server visa acelerar a compreensão do fluxo de dados e facilitar a implementação contínua de funcionalidades.
- **Centralização e Integridade de Dados:** O sistema deve garantir que todas as regras de negócio e validações de segurança ocorram no servidor. O objetivo é manter a consistência das informações apresentadas aos múltiplos clientes e evitar a exposição de lógicas sensíveis no lado do navegador.

2.4.2 Restrições

- **Padrão Arquitetural Mandatório:** O sistema deve ser obrigatoriamente construído sobre o padrão Client-Server, baseando-se em requisições via rede (tipicamente HTTP/HTTPS) para a troca de dados entre o front-end e o back-end.
- **Stack Tecnológica:** O desenvolvimento da camada cliente (e parte do roteamento/servidor, se aplicável) está restrito ao uso do framework Next.js. As ferramentas de suporte e

bibliotecas adotadas devem possuir documentação e compatibilidade comprovadas com esse ecossistema web moderno.

- **Prazo de Entrega:** As decisões arquiteturais estão limitadas ao tempo disponível para o ciclo de desenvolvimento. Soluções de alta complexidade que não apresentem documentação ampla ou que exijam um tempo de aprendizado incompatível com o cronograma de entregas não devem ser adotadas.

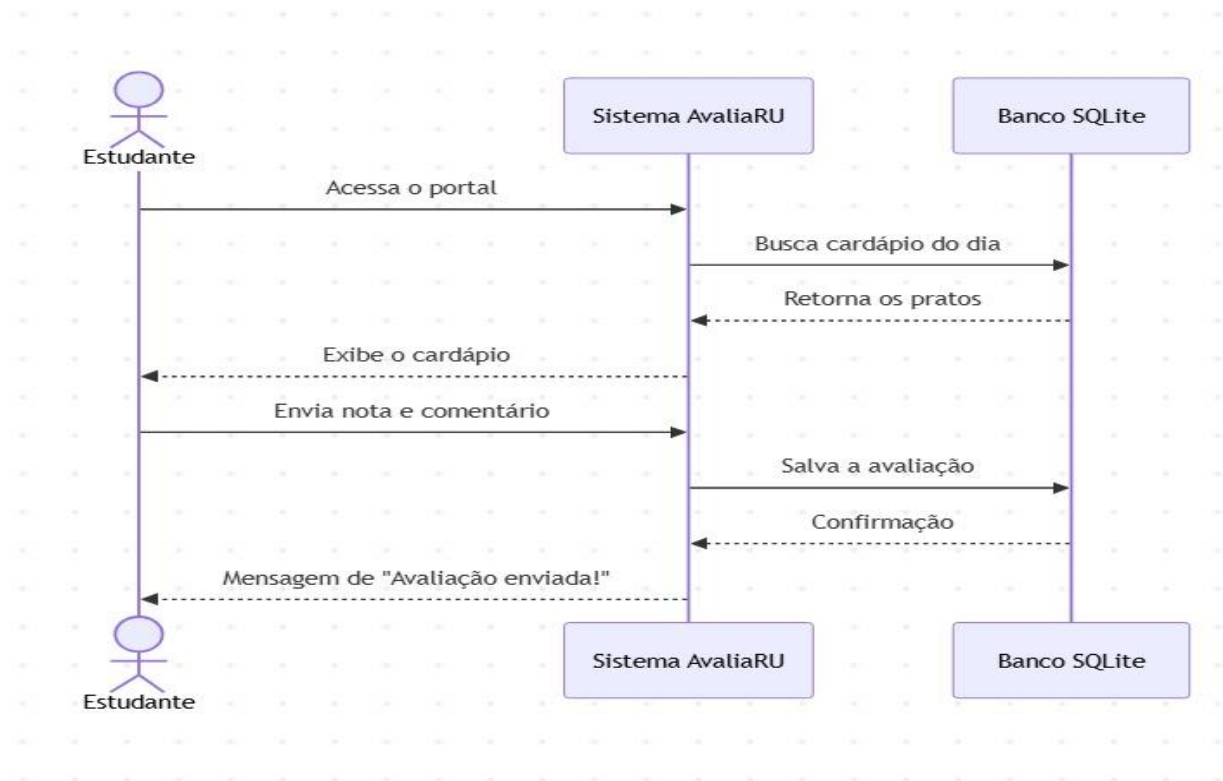
2.5 Visões

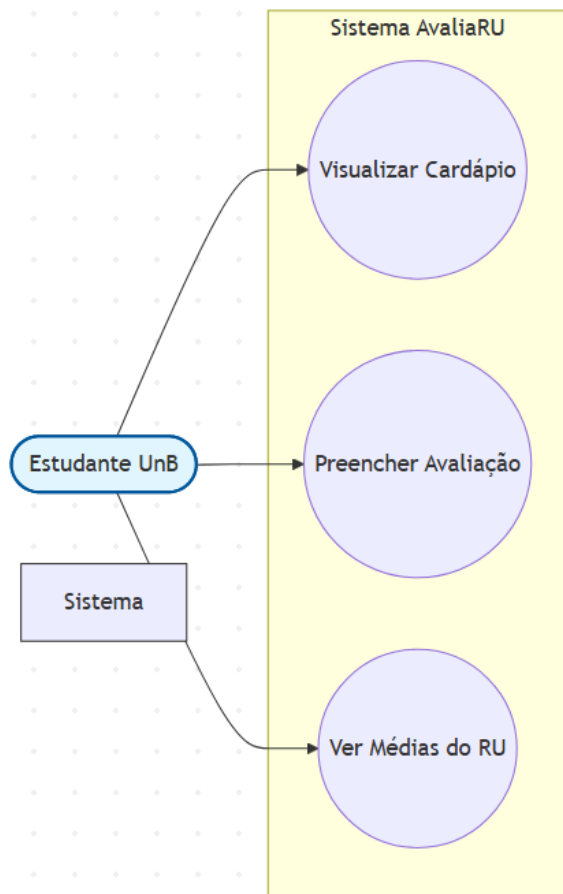
- As visões a seguir detalham a arquitetura sob diferentes perspectivas, permitindo uma compreensão completa do funcionamento do sistema.

2.5.1 Visão uso

Esta visão descreve o escopo do sistema focado no usuário. O AvaliaRU permite que o estudante visualize o cardápio e registre seu feedback. A escolha tecnológica baseou-se nos **quatro pilares de decisão arquitetural**: Requisitos Funcionais (gestão de avaliações), Requisitos Não Funcionais (responsividade via celular), Prazos (agilidade do Next.js) e Experiência da Equipe (domínio de JavaScript/TypeScript).

Diagrama de Casos de Uso:





6

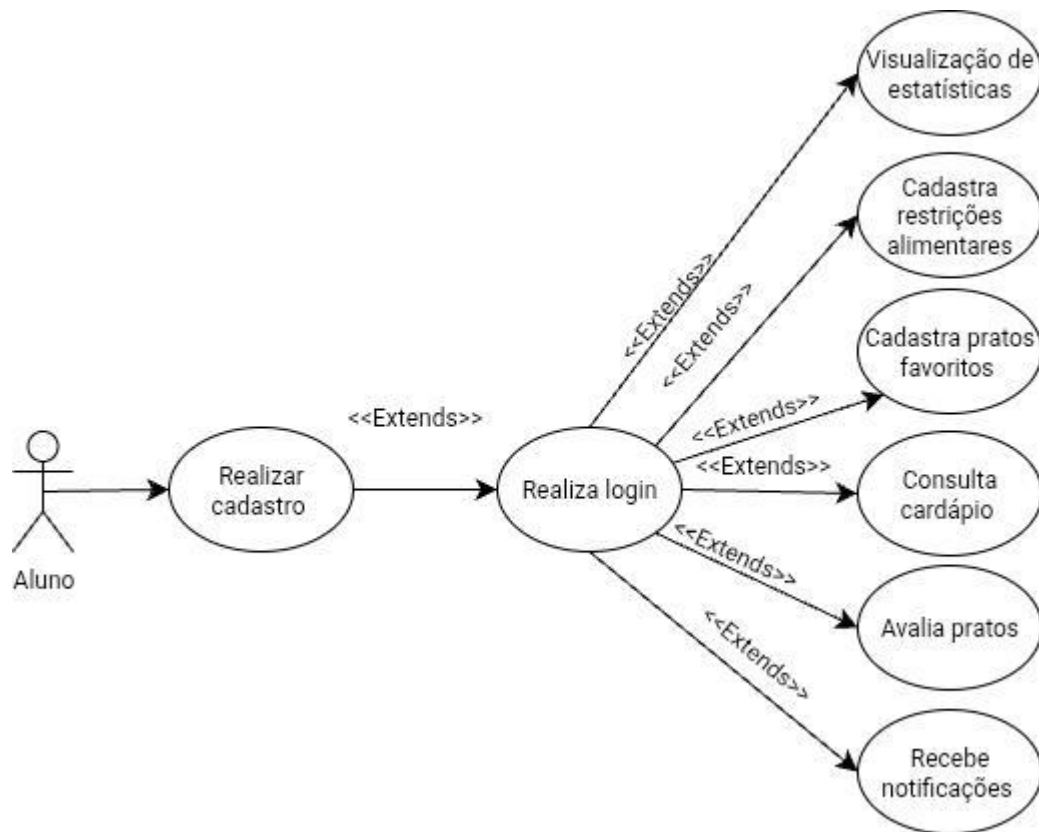
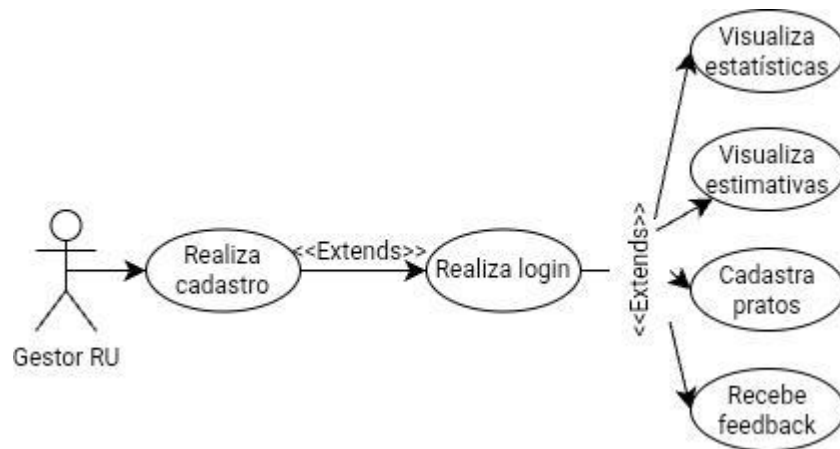
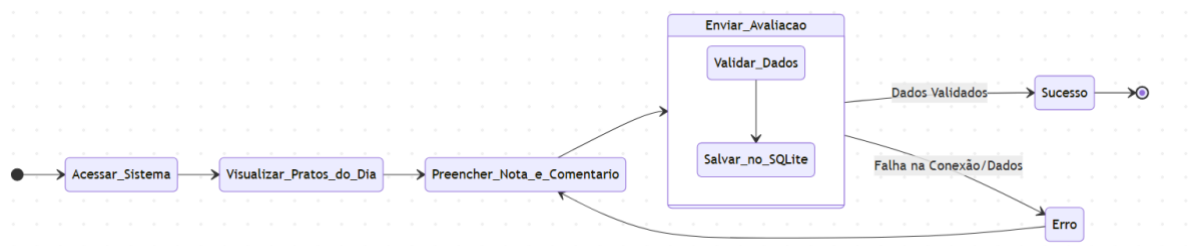


Diagrama de Atividades: Detalha o fluxo da funcionalidade de submissão de avaliação, do acesso à confirmação.

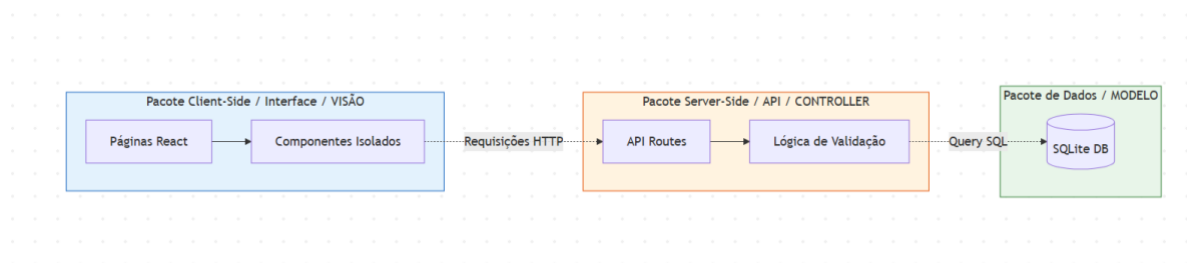


2.5.2 Visão de organização lógica

A estrutura lógica segue o padrão **MVC (Modelo-Visão-Controller)**, separando responsabilidades para facilitar a evolução do sistema:

- **Visão (Front-end):** Componentes React que gerenciam a interface e a experiência do usuário.
- **Controller (API Routes):** Rotas do Next.js que intermedeiam a comunicação, validando a lógica de negócio.
- **Modelo (Acesso a Dados):** Camada que encapsula a interação com o SGBD SQLite.

Diagrama de Pacotes:



2.5.3 Visão estrutural

Os diagramas abaixo foram elaborados sob uma **Perspectiva de Especificação/Implementação**, focando nas abstrações de software e componentes reais do projeto.

Diagrama de Classes:

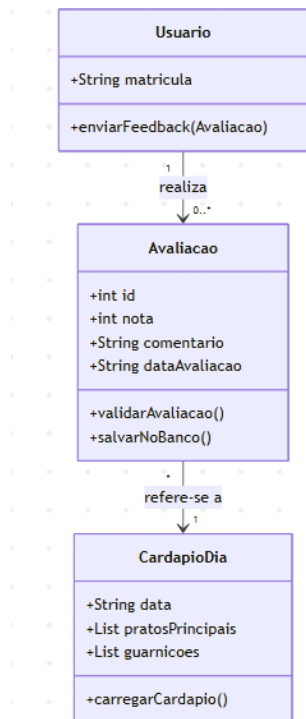
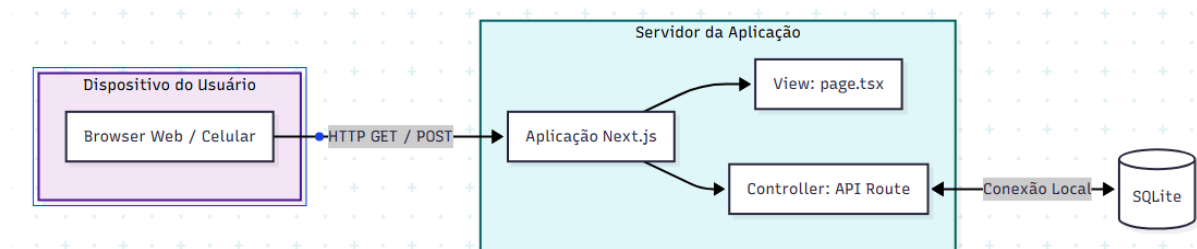


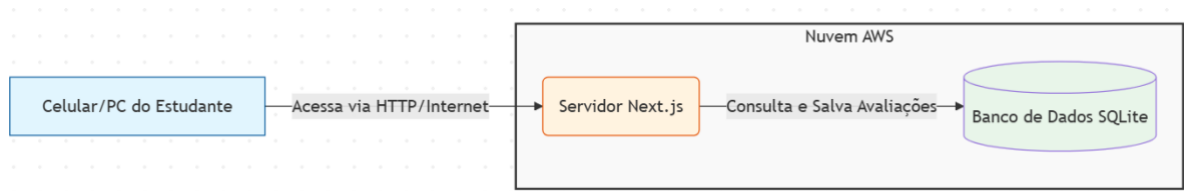
Diagrama de Componentes:



2.6 Visão de Implantação

O sistema será implantado em ambiente de nuvem utilizando a infraestrutura da **AWS (Amazon Web Services)**. O servidor Next.js lidará com as requisições e a persistência será feita em arquivo local SQLite, garantindo simplicidade e portabilidade do sistema.

Diagrama de Implantação:



2.7 Restrições adicionais

Esta seção descreve as restrições arquiteturais suplementares do sistema AvaliaRU, abrangendo regras de negócio, atributos de qualidade de software e diretrizes de segurança e controle de acesso.

2.7.1 Restrições de Negócio e Operacionais

- **Acessibilidade Web:** O sistema AvaliaRU consiste em uma aplicação Web hospedada na AWS, acessível diretamente pela Internet por meio de navegadores em computadores ou dispositivos móveis.
- **Autenticação Obrigatória:** O uso do sistema não é aberto ao público geral de forma anônima. É exigido o cadastro prévio e a autenticação (login) para que estudantes, gestores do RU e administradores possam acessar as funcionalidades e realizar operações no sistema.

2.7.2 Características de Qualidade de Software

- **Usabilidade e Responsividade (Portabilidade):** O design da interface deve ser obrigatoriamente adaptável para o uso em dispositivos móveis (*Mobile-first*). Os elementos da interface, como botões de avaliação, devem se adaptar à tela de navegadores mobile (ex: Chrome Mobile) sem quebra de layout.
- **Desempenho (Performance):** Devido à alta rotatividade e curtas janelas de tempo de pico no refeitório, o sistema deve apresentar um tempo médio de resposta menor ou igual a 2 segundos nas operações críticas, como visualizar o cardápio, salvar avaliações e registrar restrições.
- **Confiabilidade:** O sistema deve garantir a persistência e a integridade das transações mesmo sob alta carga de acessos simultâneos, evitando falhas de persistência durante

os horários de pico do restaurante universitário, utilizando o banco de dados SQLite. Além disso, o módulo de alertas deve operar com alta confiabilidade, mantendo uma taxa de sucesso igual ou superior a 98% na exibição correta das notificações personalizadas de alergênicos e favoritos.

- **Disponibilidade (Janelas de uso):** O sistema deve garantir uma disponível com alta confiabilidade durante o horário de funcionamento do RU, sendo essa janela de uso de segunda a sexta, das 6:30 até às 20:00, e Sábados das 7:00 até às 14:30. O sistema deverá apresentar 95% de uptime dentro da janela definida na semana (máximo de 3h e 45 min fora do ar).

2.7.3 Segurança e Permissões de Acesso

Para garantir a segurança das informações e a correta divisão de responsabilidades, o sistema exige controle de acesso baseado em três perfis distintos de usuários:

- **Estudante (FCTE):** É o usuário final do sistema. Possui permissão apenas para visualizar o cardápio diário, configurar suas restrições alimentares e pratos favoritos, e registrar ou editar avaliações das refeições consumidas.
- **Gestor RU:** Representa a administração do restaurante. Possui permissões para visualizar as avaliações quantitativas e qualitativas feitas pelos alunos, além de cadastrar, visualizar e remover os pratos ofertados na aplicação.
- **Administrador:** Membro da equipe técnica do AvaliaRU. Possui acesso total ao sistema, com permissões exclusivas para gerenciar contas de usuários, gerenciar os perfis de acesso e realizar rotinas de manutenção, como backups do banco de dados SQLite.

Há também a necessidade de garantir a segurança no tratamento de dados sensíveis de acordo com a LGPD, como é o caso das restrições alimentares dos estudantes. Para isso, esses dados devem ser criptografados antes de serem submetidos a camada de persistência (SQLite), sendo proibida a sua associação a dados nominais em possíveis relatórios. Esses dados sensíveis devem ser completamente eliminados do sistema mediante solicitação de exclusão de conta pelo usuário.

2.7.4 Padronização de Estrutura e Código

- **Restrição de Ambiente:** O ambiente de desenvolvimento deve ser padronizado via container (Codespaces) para garantir a paridade entre os ambientes de desenvolvimento e produção (AWS).
- **Modularização Obrigatória:** O projeto deve seguir estritamente a arquitetura de pastas do Next.js, separando as camadas de Front-end, Back-end e de Persistência de Dados. Lógicas de infraestrutura e serviços externos devem ser isoladas em módulos, sendo vedada a implementação de lógica de acesso a dados diretamente nos componentes da interface (padrão de Monolito Modular).
- **Restrição de Versionamento:** Uma alteração de código só pode ser mesclada à branch “main” ao passar por uma revisão de pares e verificação de build.

3 Bibliografia

1. BECK, K. **Programação extrema (XP) explicada**. Porto Alegre: Bookman, 2004.
2. SOMMERVILLE, I. **Engenharia de software**. 10. Ed. São Paulo: Pearson Addison Wesley, 2019.
3. SCHWABER, K; SUTHERLAND, J. **O guia do Scrum: o guia definitivo para o Scrum: as regras do jogo**. [S.L.: s.n.], 2020.
4. GABRIEL, José; NICOLAS, Rafael. **Pesquisa de opinião dos alunos**. Brasília, 2026. 1 questionário aplicado em maio de 2026.
5. VERCEL. **Next.js Documentation**. [S. I.], 2026. Disponível em: <https://nextjs.org/docs>. Acesso em: 06 de maio 2026

6. SQLITE. **SQLite Documentation**. [S. I.]: SQLite Consortium, 2024. Disponível em: <https://sqlite.org/docs.html>. Acesso em 06 de maio 2026
7. AMAZON WEB SERVICES. **AWS Documentation**. [S. I.]: Amazon, 2026. Disponível em: <https://docs.aws.amazon.com/> Acesso em 06 de maio 2026